

>_ SESSION 01 • WEB DEVELOPMENT

Course Intro & How the Web Works

Tools setup, the client-server model, and how a URL turns into a page on screen.

1

Tools & setup

2

Client-server

3

URL to page

4

HTML structure

>_ Vasken Sarkis • Course Intro & How the Web Works

Today's session

- 01 Tools setup — VS Code, browser dev tools, Git & GitHub basics
- 02 The client-server model — who asks, who answers
- 03 How a URL becomes a webpage
- 04 HTML document structure & semantic tags



ALSO TODAY

Lab: set up your dev environment, create a Git repo, and build a bare-bones HTML page.

01

Tools & Setup

The editor, the inspector, and the safety net — get all three running before we write a line of HTML.

Your toolkit

editor

VS Code

- Where you'll write every line of HTML, CSS, and JS this term
- Built-in terminal — no separate app needed
- Live Server extension for instant reloads

inspector

Browser dev tools

- Right-click → Inspect on anything
- Elements panel shows the live DOM tree
- Console surfaces errors as you build

version control

Git & GitHub

- Git tracks changes to your files over time
- GitHub hosts your repos and your portfolio
- Every assignment this term ships as a repo

Git & GitHub, the four commands

terminal

```
1 $ git init
2 $ git add .
3 $ git commit -m "first commit"
4 $ git push origin main
```



NOTE

`init` starts tracking a folder. `add` stages changes. `commit` saves a snapshot. `push` uploads it to GitHub.



TIP

Commit early, commit often — small commits are easier to undo than one giant one.

02

The Client-Server Model

Every website is a conversation between two computers — one that asks, one that answers.

>_ THE CLIENT-SERVER MODEL

Client asks, server answers



KEY IDEA

The client sends an HTTP request for something (a page, an image, a script); the server sends back an HTTP response with that thing, or an error.

03

From URL to Webpage

The four steps between typing an address and seeing a rendered page.

>_ FROM URL TO WEBPAGE

From URL to webpage

```
https://www.example.com/courses/webdev
```

protocol domain path

- 01 DNS translates the domain into a server's IP address.
- 02 The browser sends an HTTP request to that address, asking for the path.
- 03 The server finds the matching file and sends it back as HTML.
- 04 The browser parses that HTML and renders it as the page you see.

04

HTML Document Structure

Every HTML page shares the same skeleton — and semantic tags describe what each region means.

Anatomy of an HTML document

index.html

```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8">
5     <title>My Page</title>
6   </head>
7   <body>
8     <h1>Hello, web!</h1>
9   </body>
10 </html>
```



NOTE

<!DOCTYPE html> tells the browser to render in modern standards mode.



TIP

<head> holds metadata — nothing in it is visible. <body> holds everything that renders.

Semantic tags structure the page

`<header>`

`<nav>`

`<main>`

`<footer>`

header — site title, logo, top-level intro

nav — the page's primary links

main — the one unique content region of the page

footer — copyright, contact, secondary links



TIP

Semantic tags describe meaning, not appearance — screen readers and search engines rely on them even though a `<div>` would look identical.

Set up shop, build the skeleton

Get the environment running, then build a first bare-bones page.

1. Install VS Code and open the browser dev tools
2. Create a Git repo and make your first commit
3. Build an HTML page with headings and paragraphs
4. Lay it out with <header>, <nav>, <main>, <footer>

git init

semantic HTML

no styling yet

Save, commit, and push before you leave.

Finish the "About Me" skeleton

HTML only

no CSS yet

single page

- Finish the personal "About Me" page started in the lab
- Use semantic layout tags — header, nav, main, footer
- Include at least one heading and two paragraphs
- Commit and push to your GitHub repo



DUE

Start of Session 2. We'll add lists, tables, and a contact form on top of it next class.